Theoretically Principled Deep RL Acceleration via Nearest Neighbor Function Approximation

Junhong Shen, Lin Yang
University of California, Los Angeles
jhshen@ucla.edu, linyang@ee.ucla.edu

Abstract

Recently, deep reinforcement learning (RL) has achieved remarkable empirical success by integrating deep neural networks into RL frameworks. However, these algorithms often require a large number of training samples and admit little theoretical understanding. To mitigate these issues, we propose a theoretically principled nearest neighbor (NN) function approximator that can replace the value networks in deep RL methods. Inspired by human similarity judgments, the NN approximator estimates the action values using rollouts on past observations and can provably obtain a small regret bound that depends only on the intrinsic complexity of the environment. We present (1) Nearest Neighbor Actor-Critic (NNAC), an online policy gradient algorithm that demonstrates the practicality of combining function approximation with deep RL, and (2) a plug-and-play NN update module that aids the training of existing deep RL methods. Experiments on classical control and MuJoCo locomotion tasks show that the NN-accelerated agents achieve higher sample efficiency and stability than the baseline agents. Based on its theoretical benefits, we believe that the NN approximator can be further applied to other complex domains to speed-up learning.

Introduction

People learn a variety of relationships in life, e.g., we associate the force of pressing the gas pedal with the amount of acceleration gained while driving. In the context of reinforcement learning (RL), where an agent interacts with the environment to maximize the cumulative rewards, the learning objective is the relationship between state-action pairs and future gains. Theories on associative learning suggest that people learn from similarity measures (Carroll 1963; Busemeyer et al. 1997): if x can predict y, they presume that observations similar to x have similar y values. It is thus natural to consider integrating similarity-based models into active learning. A suitable choice for the RL setting is the nearest neighbor function approximator (Emigh et al. 2016; Shah and Xie 2018; Yang, Ni, and Wang 2019).

We study online episodic RL with unknown reward and transition functions. Existing deep RL algorithms achieve impressive results in robot control (e.g., Lillicrap et al. 2016; Levine et al. 2016; Gu et al. 2017), Go (Silver et al. 2016) and Atari playing (Mnih et al. 2013). However, several challenges still exist. First, the theoretical foundation of deep RL has not been fully established (Arulkumaran et al. 2017; Lake et al. 2018). It is often mysterious why an algorithm works or fails in certain cases (Kansky et al. 2017). Second, empirical results suggest that model-free deep RL often requires considerable samples to learn (Deisenroth and Rasmussen 2011). Nonetheless, data can be expensive to acquire in practical domains like healthcare (Kober, Bagnell, and Peters 2013). High-dimensional input, such as pixel data, demands a larger sample size even if the problem itself is simple (Lillicrap et al. 2016). Third, online learning coupled with neural networks is generally regarded as unstable (van Hasselt, Guez, and Silver 2016). Hyperparameter tuning can also affect the learning outcome. In sum, improving deep RL with theory-based approaches is of critical importance.

On the other hand, great theoretical progress has been made in tabular RL, where the state-action spaces are small and finite. The sample complexity of tabular methods are properly studied (e.g., Jaksch, Ortner, and Auer 2008; Azar, Osband, and Munos 2017; Jin et al. 2018; Zanette and Brunskill 2019). Yet the best obtainable complexity depends linearly on the number of states, which can be huge in reality, e.g., the game of Go has $3^{19\times 19}$ states. Thus, real-world application of tabular theories remains a challenge.

In this paper, we bridge the gap between RL theory and practice by a theoretically principled deep RL acceleration technique. Specifically, we exploit the structural properties of the state-action space by using the nearest neighbor (NN)[1] approximator for value estimation. Such a function approximator not only attains a theoretical guarantee in sample complexity but also possesses good generalization ability when plugged into deep RL frameworks. In fact, we show that the NN approximator with an upper-confidence construction obtains a near-optimal regret $O[H(DLK)^{d/(d+1)}]$ for both low- and high-dimensional inputs in deterministic systems, where K is the number of episodes, H is the episode length, L is a Lipschitz constant related to the distance metric measuring state similarities, D and d are respectively the diameter and dimension of the intrinsic state-action space.

We demonstrate the empirical efficacy of the NN approx-

Copyright © 2021, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

[1]Throughout the paper, the abbreviation "NN" is used to refer to "nearest neighbor," not "neural network."

imator by fitting it into the actor-critic framework. We use the non-parametric NN critic to bootstrap state values and train the policy network with temporal-difference methods (Sutton 1988). Given a state-action pair, the NN critic finds within history the closest sample to this observation under the distance metric. The corresponding reward plus a Lipschitz confidence term and the next state are used to approximate the reward and transition functions, respectively. The algorithm displays impressive learning speed in the cart-pole balancing problem. Beyond this, we also encapsulate the NN approximator into a plug-and-play module that boosts the training of existing deep RL algorithms without changing their original structures. The plug-in NN critic encourages action exploration and stabilizes value learning. We evaluate the module with state-of-the-art deep RL agents on a set of 3D locomotion tasks. Results show that the NN-aided gradient update improves both training speed and stability.

Roadmap: The paper is organized as follows: we first discuss related works in theoretical and deep RL. Next, we introduce notations and concepts in RL and metric dimensions. We then give the formulation of the NN approximator and analyze its theoretical guarantee. In the later sections, we present two algorithms that combine the NN approximator with deep RL and evaluate their empirical performance.

Related Work

RL with neural networks: There is a long line of research that applies deep RL to games and control problems (e.g., Mnih et al. 2013; Schulman et al. 2015; Levine et al. 2016; van Hasselt, Guez, and Silver 2016; Lillicrap et al. 2016). These results employ several heuristics to accelerate exploration. For example, Mnih et al. (2013, 2015); van Hasselt, Guez, and Silver (2016) randomly sample actions and store them in a replay buffer before policy learning. Gaussian noise (Lillicrap et al. 2016) or Ornstein-Uhlenbeck noise (Fujimoto et al. 2018) are added to the actions or the network parameters (Plappert et al. 2018) to encourage exploration. Several works also combine model-based value estimation and model-free policy learning to reduce sample complexity (McAllister and Rasmussen 2016; Buckman et al. 2018).

Although there is limited theoretical understanding of the aforementioned methods, we emphasize that our aim is not to improve them but rather introduce a new theory-based function approximator to the literature of deep RL. By combining NN value estimation and existing frameworks, we believe that the efficiency of model-free RL algorithms can be improved in a provable manner (at least in some settings).

RL with theoretical guarantees: To facilitate theoretical analysis, many works study RL in the tabular setting, where the state and action spaces are discrete (e.g., Jaksch, Ortner, and Auer 2008; Azar, Osband, and Munos 2017; Jin et al. 2018; Zanette and Brunskill 2019). The sample complexity of the algorithms depends at least linearly on the number of states $|\mathcal{S}|$. Since this number tends to be large in practice, it is difficult to extend these algorithms to real-world settings. Recently, several works have emphasized understanding in RL with general function approximation (e.g., Osband and Roy 2014; Jiang et al. 2017; Sun et al. 2019; Wang et al. 2020). However, the function classes are either simple, e.g., linear functions (e.g., Yang and Wang 2019; Jin et al. 2020), or have strong structural assumptions, which prevent them from being applied to more practical problems.

RL with nearest neighbor search: Combining nearest neighbor search with active learning has been studied in episodic RL. Model-free episodic control (Blundell et al. 2016) builds on a tabular memory and applies regression using the mean of k-nearest neighbors for Q-value estimation. Neural episodic control (Pritzel et al. 2017) and episodic memory deep Q-networks (Lin et al. 2018) improve the algorithm's generalization ability by absorbing state features into networks. These algorithms take the NN search as a pure classification technique. They do not exploit the fact that the distance between state-action pairs can indicate their relative values. In addition, the value estimation for these methods exists at the trajectory level: the Q-values are updated at the end of an episode by the total reward of a trajectory. In contrast, the NN value estimation in this paper takes the form of on-policy Monte Carlo rollouts using samples from independent environment steps. Thus, we leverage not only intra-episode but also inter-episode information.

The prototype of our NN function approximator is presented in Yang, Ni, and Wang (2019), where an upper-confidence algorithm with general function approximation is proposed for tabular RL. The algorithm can apply to continuous cases but requires a discretization of the action space. We modify the original algorithm to account for non-tabular cases without discretizing the action space. Though Yang, Ni, and Wang (2019) derive a regret based on the ambient dimension of the state space, they do not justify the regret of high-dimensional inputs with small intrinsic dimensions.

Preliminaries

In this section, we introduce the key definitions and notations in RL. For the clarity of the proofs, we assume that the Markov decision process (MDP) is finite-horizon and deterministic. This assumption is not restrictive, as many real-world control systems do not involve randomness.

Formally, we consider an MDP $(\mathcal{S}, \mathcal{A}, f, r, H)$ with state space $\mathcal{S}$, action space $\mathcal{A}$, deterministic transition model $f : \mathcal{S} \times \mathcal{A} \to \mathcal{S}$, and reward function $r : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$. An agent interacts with the environment episodically, where each episode lasts H steps. In an episode, the agent starts from an initial state s_1 independent of the history. At step $h \in [H]$[2], it observes state s_h and performs action $a_h := \pi(s_h, h)$ according to the policy $\pi : \mathcal{S} \times [H] \to \mathcal{A}$. It then receives reward $r_h = r(s_h, a_h)$ and next state $s_{h+1} = f(s_h, a_h)$. We define the cumulative reward from h as $R_h = \sum_{t=0}^{H-h} r_{h+t}$. The goal of learning is to find a policy that maximizes the total reward in one episode when f and r are unknown.

Given a policy π, the value function $V^\pi : \mathcal{S} \times [H] \to \mathbb{R}$ is defined as the cumulative reward from state s at step h and following π therefrom. It satisfies the Bellman equation:

$$V_h^\pi(s) = r(s, \pi(s)) + V_{h+1}^\pi[f(s, \pi(s))], \qquad (1)$$

[2] $[H]$ denotes the set of integers $\{1, ..., H\}$.

with $V_H^\pi(s) = r(s, \pi(s))$. The optimal policy π^* is the one such that $V_h^{\pi^*} := V_h^* = \max_\pi V_h^\pi$. The temporal-difference (TD) error at step h is:

$$\delta_h = r_h + V_{h+1}^\pi(s_{h+1}) - V_h^\pi(s_h). \tag{2}$$

We further denote the action value (or Q-function) as $Q_h^\pi(s,a) := r(s,a) + V_{h+1}^\pi\big(f(s,a)\big)$. The optimal $Q_h^* := \max_\pi Q_h^\pi$ gives the maximum values for a (s,a) pair achievable by any policy. By the Bellman optimality equation, we have $V_h^*(s) = \max_{a\in A} Q_h^*(s,a)$.

We measure the sample complexity of an algorithm by its regret, which is the difference between the total rewards of the unknown optimal policy and that gathered in learning:

$$\mathrm{Regret}(K) = \sum_{k=1}^{K}[V_1^*(s_1^k) - \sum_{h=1}^{H} r(s_h^k, a_h^k)]$$

where K is the number of episodes played.

Theoretical Guarantee of Nearest Neighbor Function Approximation

In this section, we first introduce concepts relevant to the structure of an MDP. We then formalize the nearest neighbor function approximator and show its theoretical guarantee in terms of sample complexity.

Metric Space and Intrinsic Dimension

In practice, the state-action space $\mathcal{X} = \mathcal{S} \times \mathcal{A}$ is usually continuous. We assume that $\mathcal{X}$ is a metric space with a distance function $d_\mathcal{X}$ that satisfies the triangular inequality. This assumption is easily achievable, e.g., the Euclidean distance can be applied to a space of pixel data. We also assume that $\mathcal{X}$ is bounded and has diameter $D := \sup_{x,x'\in\mathcal{X}} d_\mathcal{X}(x,x')$.

An intuitive way to measure the complexity of $\mathcal{X}$ is through the ambient dimension p, which can be roughly understood as the number of variables *used* to describe a point in $\mathcal{X}$. In real-world MDPs, the states are often represented by real-valued vectors, which form a Euclidean ambient space. Thus, in our context, we simply take p as the Euclidean dimension of the *natural* embedding of $\mathcal{X}$. For instance, a 20×20 image has $p = 400$ regardless of its content.

However, most meaningful, high-dimensional data do not uniformly fill in the space where they are represented. Rather, they concentrate on smooth manifolds with low *intrinsic* dimension. The intrinsic dimension d measures the inherent complexity of a metric space. Informally, it is the number of variables *needed* to describe $\mathcal{X}$. Suppose the aforementioned image depicts a car with 5 physical properties, then d can be 5 rather than 400. Studies on intrinsic dimension estimation (e.g., Kégl 2002; Levina and Bickel 2004) employ more formal definitions. Yet the technical details are out of our scope. We only require $d \le p$ in general.

For the clarity of later proofs, we outline two concepts that can bound the intrinsic dimension of a metric space.

Definition 1 (Covering and packing)**.** *An ϵ-cover of a metric space $\mathcal{X}$ is a subset $\hat{\mathcal{X}} \subseteq \mathcal{X}$ such that for each $x \in \mathcal{X}$, there exists $x' \in \hat{\mathcal{X}}$ with $d_\mathcal{X}(x,x') \le \epsilon$. The ϵ-covering number is $N(\epsilon) = \min\{|\hat{\mathcal{X}}| : \hat{\mathcal{X}}$ is an ϵ-cover of $\mathcal{X}\}$. An ϵ-packing is $\hat{\mathcal{X}} \subseteq \mathcal{X}$ such that $\forall x, x' \in \hat{\mathcal{X}}$, $d_\mathcal{X}(x,x') > \epsilon$. The ϵ-packing number is $M(\epsilon) = \max\{|\hat{\mathcal{X}}| : \hat{\mathcal{X}}$ is an ϵ-packing of $\mathcal{X}\}$.*

Nearest Neighbor Function Approximator

In the RL setting, the learning algorithm collects a set of observations $B := \{x_i\}_{i\in[N]}$ and corresponding value labels $\{Q(x_i)\}_{i\in[N]} \subseteq \mathbb{R}$. The unknown $Q : \mathcal{X} \to \mathbb{R}$ measures the quality of a state-action pair. As in supervised machine learning, the major task is to find a function approximator $f : \mathcal{X} \to \mathbb{R}$ that approximates the known labels with small errors and also generalizes to unseen data points. That is, $f(x_i) \approx Q(x_i)$ for all $i \in [N]$ and $f(x) \approx Q(x)$ for $x \in \mathcal{X}\backslash B$. With the distance metric $d_\mathcal{X}$, we can now define the NN approximator which satisfies the above property.

Definition 2 (**Nearest neighbor function approximator**)**.** *Given a sample buffer $B = \{\big(x_i, Q(x_i)\big)\}_{i\in[N]} \subseteq \mathcal{X} \times \mathbb{R}$, the nearest neighbor approximator is the function $\hat{Q} : \mathcal{X} \to \mathbb{R}$ such that $\forall x \in \mathcal{X}$,*

$$\hat{Q}(x) := \min_{i\in[N]}\{Q(x_i) + L \cdot d_\mathcal{X}(x, x_i)\}.$$

$L > 0$ is a parameter that adjusts the approximation error.

Note that $\hat{Q}(x)$ matches existing samples exactly and the approximation error for a new x is characterized by an upper bound obtained from the closest known data. This contrasts with other function approximators that lack theoretical understanding, e.g., neural networks. Now, we proceed to show more practical guarantees of the NN function approximator.

MDP with Lipschitz Continuity

To ensure the problem is tractable, we impose the following regularity condition on the optimal Q-function of the MDP.

Assumption 3 (MDP with Lipschitz continuity)**.** *In the metric space $\mathcal{X}$, let the optimal Q-function be $Q_h^* : \mathcal{X} \to \mathbb{R}$, then $\exists L_1, L_2 > 0$ such that $\forall x, x' \in \mathcal{X}$, $\forall h \in [H]$,*

$$|Q_h^*(x) - Q_h^*(x')| \le L_1 \cdot d_\mathcal{X}(x,x'), \tag{3}$$

$$\max_{a''} d_\mathcal{X}\big[\big(f(x), a''\big), \big(f(x'), a''\big)\big] \le L_2 \cdot d_\mathcal{X}(x,x'). \tag{4}$$

Assumption 3 implies that there is a proper notion of distance in $\mathcal{X}$ such that two points close to each other have similar Q-values. It also ensures that the system is stable. Let $L = L_1(L_2 + 1)$ be the parameter defining $\hat{Q}(x)$, then $\hat{Q}(x)$ is L-Lipschitz continuous (Yang, Ni, and Wang 2019, Lemma 2).

Yang, Ni, and Wang (2019) proposed an upper-confidence algorithm with NN approximation (UCRL-FA) for discrete deterministic MDPs. Basically, the approximate Q_h is updated recursively from $h = H$ to $h = 1$ at the end of each episode. The agent acts according to the greedy policy $\arg\max_{a\in\mathcal{A}} Q_h$. For completeness, we present UCRL-FA in Appendix A. It achieves the following regret bound.

Theorem 4 (Regret till ϵ-optimality by Yang et al)**.** *Suppose $\mathcal{X}$ admits an ϵ-cover with size $N(\epsilon)$ for any $\epsilon > 0$. After K episodes, UCRL-FA with the nearest neighbor construction obtains a regret bound $Regret(K) \le HN(\epsilon) + 2\epsilon LKH$. If $\mathcal{X}$ is compact, then $Regret(K) = O(DLK)^{\frac{p}{p+1}} \cdot H$.*

For a metric space with ambient dimension p, Theorem 4 states that UCRL-FA learns the system with ϵ^{-p} samples, where ϵ is the learning accuracy. The algorithm is efficient for low-dimensional observations. Indeed, in real-world MDPs governed by laws of physics, the intrinsic dimensions are usually small. However, the internal states are not always available. If there are only pixel data, p can be hundreds if not thousands even for a simple system like a moving car. The natural question is: *can the NN function approximator achieve a regret that depends only on the intrinsic dimension d?* In the next section, we answer the question affirmatively with some mild assumptions.

Efficient Value Learning in Metric Spaces

To characterize the true complexity of the NN approximator, we additionally distinguish the state-action spaces. Suppose the learner observes a p-dimensional state space $\hat{\mathcal{S}}$, e.g., image space, which admits an embedding to a d-dimensional space $\mathcal{S}$, e.g., the parameter space of a physical system, for some $d \ll p$. We emphasize that the learner does not have any information about $\mathcal{S}$. Let $\mathcal{Y} = \hat{\mathcal{S}} \times \mathcal{A}$ and $d_{\mathcal{Y}}$ be the distance metric satisfying the triangular inequality. Since the action space remains the same, we simply denote the ambient dimension of $\mathcal{Y}$ as p and the intrinsic dimension as d.

The relationship between any true state s and its external representation $\hat{s}$ can be described by a function that maps $\mathcal{S}$ to $\hat{\mathcal{S}}$. In our proofs, we only assume the existence of the mapping, but do not require knowing its explicit form.

Assumption 5 (Bi-Lipschitz mapping between metric spaces)**.** *There exists a map $g : \mathcal{X} \to \mathcal{Y}$ from the intrinsic state space to the external metric space. We assume that g is Bi-Lipschitz, i.e., $\forall x, x' \in \mathcal{X}$, $\exists C > 0$ such that*

$$C^{-1} d_{\mathcal{X}}(x, x') \leq d_{\mathcal{Y}}(g(x), g(x')) \leq C d_{\mathcal{X}}(x, x'). \quad (5)$$

$d_{\mathcal{X}}$ and $d_{\mathcal{Y}}$ are distance metrics for $\mathcal{X}$ and $\mathcal{Y}$, respectively.

Assumption 5 models real-world RL systems: if two points are close in the observation space, they are close internally. Now, by Assumption 3, we obtain the following lemma on the continuity of the observation space.

Lemma 6 (High-dimensional MDP with Lipschitz continuity)**.** *If $\mathcal{X}$ satisfies Assumption 3 and $g : \mathcal{X} \to \mathcal{Y}$ satisfies Assumption 5, then the MDP with state-action space $\mathcal{Y}$ is $\hat{L}$-Lipschitz continuous, where $\hat{L} = (L_2 C^2 + 1) \cdot L_1 C$.*

The formal proof can be found in Appendix B. Now, we can treat $\mathcal{Y}$ as an independent MDP without knowing its intrinsic properties and directly apply Theorem 4. Lemma 7 characterizes the regret of the NN approximator in $\mathcal{Y}$.

Lemma 7 (Regret in $\mathcal{Y}$ w.r.t. the local dimension)**.** *Suppose that $\mathcal{Y}$ admits an $\hat{\epsilon}$-cover $N(\hat{\epsilon})$ for some $\hat{\epsilon} > 0$, after K episodes, the NN function approximator obtains a regret*

$$Regret(K) \leq H \cdot N(\mathcal{Y}, \hat{\epsilon}) + 2\hat{\epsilon}\hat{L}KH \quad (6)$$

where $\hat{L} = (L_2 C^2 + 1) \cdot L_1 C$.

To bound the regret of the NN approximator in the observation space, it remains to bound the covering size of $\mathcal{Y}$. Recall the covering and packing of a metric space in Definition 1, we now present the main theorem.

Theorem 8 (**Regret till $\hat{\epsilon}$-optimality for $\mathcal{Y}$ w.r.t. the intrinsic dimension**)**.** *Suppose $\mathcal{X}$ admits an ϵ-cover $N(\mathcal{X}, \epsilon)$ and $g : \mathcal{X} \to \mathcal{Y}$ satisfies Lemma 6, then the following holds:*

1. *$\mathcal{Y}$ admits an $\hat{\epsilon}$-cover with $\hat{\epsilon} = 2C\epsilon$, $N(\mathcal{Y}, \hat{\epsilon}) \leq N(\mathcal{X}, \epsilon)$*
2. *After K episodes, the NN function approximator obtains a regret bound $O(D\hat{L}'K)^{\frac{d}{d+1}} \cdot H$, where $\hat{L}' := C\hat{L}$.*

Proof. We make use of the facts that

$$\forall \epsilon > 0, M(2\epsilon) \leq N(\epsilon) \leq M(\epsilon) \quad (7)$$

and for a metric space with diameter D and intrinsic dimension d, there exists a ϵ-cover with size

$$N(\epsilon) = \Theta\left(D/\epsilon\right)^d. \quad (8)$$

We want to show that the covering number of $\mathcal{Y}$ cannot be greater than the covering number of $\mathcal{X}$ when $\hat{\epsilon}$ is set to $2C\epsilon$. Then, the regret bound in (6) can be replaced by an upper bound which depends on the inherent properties of $\mathcal{Y}$.

Note that finding $M(\epsilon)$ for a dataset $B_n = \{x_1, ..., x_n\}$ is equivalent to finding the cardinality of a maximum independent set $MI(G_\epsilon)$ in the graph $G_\epsilon(V, E)$ with vertex set $V = B_n$ and edge set $E = \{(x_i, x_j) | d(x_i, x_j) < \epsilon\}$.

Now, consider the graph $G^{\mathcal{X}}_{2\epsilon}$ constructed by the above rule. Any two connected points x, x' in the graph satisfy $d_{\mathcal{X}}(x, x') < 2\epsilon$. Denote the image of a maximum independent set $MI_{\mathcal{X}}(G^{\mathcal{X}}_{2\epsilon})$ in $\mathcal{Y}$ as $MI_{\mathcal{Y}}(G^{\mathcal{X}}_{2\epsilon})$. $MI_{\mathcal{Y}}(G^{\mathcal{X}}_{2\epsilon})$ is still a maximum independent set w.r.t. the graph with vertex set $V = B_n$ and edge set $E_{\mathcal{X}} = \{(g(x_i), g(x_j)) | d_{\mathcal{X}}(x_i, x_j) < 2\epsilon\}$ in the new metric space.

To find the packing number of $\mathcal{Y}$, we require the cutoff condition for $E_{\mathcal{Y}}$ to be $d_{\mathcal{Y}}[g(x_i), g(x_j)] < 2C\epsilon$. By (5), $d_{\mathcal{Y}}[g(x), g(x')] < 2C\epsilon$ for all elements in $E_{\mathcal{X}}$. Therefore, $E_{\mathcal{X}} \subseteq E_{\mathcal{Y}}$. In other words, $g(G^{\mathcal{X}}_{2\epsilon})$ is a subgraph of $G^{\mathcal{Y}}_{2C\epsilon}$ with the same vertices. Thus, $MI_{\mathcal{Y}}(G^{\mathcal{Y}}_{2C\epsilon}) \leq MI_{\mathcal{X}}(G^{\mathcal{X}}_{2\epsilon})$. This is because adding edges can only reduce (or remain) the size of the maximum independent sets in a graph.

Thus, $M(\mathcal{Y}, 2C\epsilon) \leq M(\mathcal{X}, 2\epsilon)$. Using (7), we conclude that $N(\mathcal{Y}, 2C\epsilon) \leq M(\mathcal{Y}, 2C\epsilon) \leq M(\mathcal{X}, 2\epsilon) \leq N(\mathcal{X}, \epsilon)$, where the first inequality holds in space $\mathcal{Y}$, the second inequality comes from the proofs above, and the last inequality holds in space $\mathcal{X}$. Preserving only the first and the last terms, we have $N(\mathcal{Y}, \hat{\epsilon}) \leq N(\mathcal{X}, \epsilon)$.

Consequently, the regret upper bound in (6) becomes

$$\begin{aligned} Regret(K) &\leq H \cdot N(\mathcal{Y}, \hat{\epsilon}) + 2\hat{\epsilon}\hat{L}KH \\ &\leq H \cdot N(X, \epsilon) + 4C\hat{L}\epsilon KH. \end{aligned}$$

In $\mathcal{X}$, equation (8) indicates that $N(\mathcal{Y}, \hat{\epsilon}) \leq \Theta(D/\epsilon)^d$, where d and D are the intrinsic properties of the state-action space. As a result, $Regret(K) \leq H \cdot \Theta(D/\epsilon)^d + 4C\hat{L}\epsilon KH$.

Denote $\hat{L}' := C\hat{L}$ as the smoothing constant for the high-dimensional metric space, when $\epsilon = D^{\frac{d}{d+1}} \cdot (CLK)^{-\frac{1}{d+1}}$, the upper bound becomes $O(D\hat{L}'K)^{\frac{d}{d+1}} \cdot H$ as desired. □

The main takeaway is that the regret in the complex space is also sub-linear in K and linear in H. Moreover, it depends on the intrinsic dimension rather than the ambient dimension. In practice, the NN approximator makes learning from images as efficient as from the actual state descriptors by emphasizing the internal differences between observations.

Algorithm 1 Nearest Neighbor Actor-Critic

Initialize experience buffer $B^1 = \emptyset$, policy parameter θ_{π^1}
for $k = 1, ..., K_{max}$ **do**
  Receive initial state s_1^k
  for $h = 1, ..., H$ **do**
    Take action a_h^k according to policy $\pi^k(a_h^k|s_h^k, \theta_{\pi^k})$
    Receive $r_h^k \leftarrow r(s_h^k, a_h^k)$ and $s_{h+1}^k \leftarrow f(s_h^k, a_h^k)$
    $\hat{V}(s_h^k) \leftarrow$ NNFUNCAPPROX$(s_h^k, h, \pi^k, B^k, H)$
    $\hat{V}(s_{h+1}^k) \leftarrow$ NNFUNCAPPROX$(s_{h+1}^k, h{+}1, \pi^k, B^k, H)$
    $\delta_h^k = r(s_h^k, a_h^k) + \gamma \cdot \hat{V}(s_{h+1}^k) - \hat{V}(s_h^k)$
    $B^k \leftarrow B^k \cup \{(s_h^k, a_h^k, f(s_h^k, a_h^k), r(s_h^k, a_h^k), \delta_h^k)\}$
    Sample a random mini-batch of N transitions from B
    Update policy with TD error policy gradient by Eq. (9)
  end for
  $B^{k+1} \leftarrow B^k$
end for

procedure NNFUNCAPPROX(s, h, $\pi(\cdot|\theta_\pi)$, B, H)
  // Assume NN look-up size M
  if $h == H$ **then**
    return 0
  $\mathcal{V} = \emptyset$
  $a \leftarrow \pi(s|\theta_\pi)$
  for $i = 1, ..., M$ **do**
    $(s_i, a_i) \leftarrow i$-th neighbor of (s, a) in B under metric d
    $r_i \leftarrow r(s_i, a_i)$, $s_i' \leftarrow f(s_i, a_i)$ ▷ stored in B
    $\hat{V}_i' \leftarrow$ NNFUNCAPPROX$(s_i', h+1, \pi(\cdot|\theta_\pi), B, H)$
    $\hat{V}_i \leftarrow r_i + \gamma \cdot \hat{V}_i' + L \cdot d[(s, a), (s_i, a_i)]$
    $\mathcal{V} \leftarrow \mathcal{V} \cup \{\hat{V}_i\}$
  end for
  return $\min \mathcal{V}$

Nearest Neighbor Actor-Critic

In this section, we integrate the NN approximator into the actor-critic framework and evaluate its practicality. This Nearest Neighbor Actor-Critic (NNAC) algorithm combines a policy network and an NN critic to solve RL problems. Upon receiving a new observation, the NN critic finds a sequence of past (s, a) pairs as a simulated trajectory and sums up the upper-bounded rewards as the value estimate. The 1-step TD error is obtained from consecutive state values. Then, the actor adjusts its policy based on the log action probability scaled by the TD error. The pseudocode is given in Algorithm 1.

Neural-Network-Based Actor

Unlike the aforementioned tabular methods which employ the greedy policy $\pi(s) = \arg\max_a \hat{Q}(s, a)$, we use a separate network parameterized by θ_π to guide the actor's movement. Let the actor loss be $J(\theta_\pi)$. The network is updated through the standard TD error policy gradient (Sutton 1988) with mini-batch size N:

$$\nabla_{\theta_\pi} J = N^{-1} \sum \delta_i \nabla_{\theta_\pi} \log \pi(a_i|s_i, \theta_\pi). \qquad (9)$$

At step h, the action distribution at s_h is pushed towards a_h if the TD error $\delta_h > 0$. We use temporal difference learning instead of directly maximizing the values to reduce variance.

Nearest-Neighbor-Based Critic

The NN critic in Algorithm 1 maintains an experience buffer B to construct the nearest neighbor tree. As in (Schulman et al. 2016), we use a parameter $\gamma \in (0, 1]$ to down-weight future rewards due to delayed effects. This parameter corresponds to the discount factor in infinite-horizon discounted problems, but we take it as a variance reduction technique.

Following previous notation, let x_h denote a state-action pair at step h. Let M be the number of neighbors considered in value approximation. The exact estimation procedure exploits Monte Carlo tree search. Given x_h, find M samples in history with the minimum distances $d_{\mathcal{X}}(x_h, x_i)$, $i \in [M]$. Consider their next states and the actions that would be taken under π_h. Expand these new state-action pairs until the tree depth reaches $H - h$. The total reward is back-propagated from the terminal states and the minimum value is the estimate for $V^\pi(x_h)$. When $M = 1$, the recursive formula is:

$$\hat{V}^\pi(x) = r(x) + \gamma \cdot \hat{V}^\pi(x') + L \cdot d_{\mathcal{X}}(x, x'). \qquad (10)$$

$x' = \arg\min_{x' \in B} d_{\mathcal{X}}(x, x')$ is the nearest neighbor and $\hat{V}^\pi(x') = \hat{Q}^\pi\big(x', \pi(x')\big)$. In long-horizon problems, we can replace the varying $H - h$ with a fixed planning horizon H'.

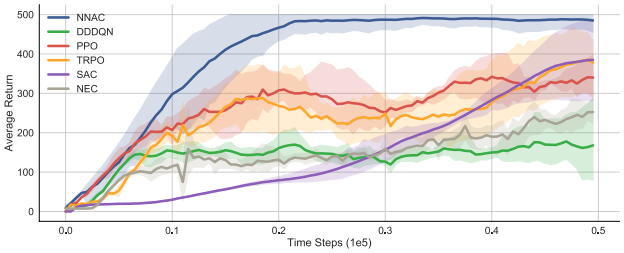
Equation (10) prioritizes exploring (s, a)'s that are farther away from the seen ones. By Assumption 3, the Lipschitz bonus $L \cdot d_{\mathcal{X}}(x, x')$ ensures that the real value of $V^\pi(x)$ is upper-bounded by $\hat{V}^\pi(x)$. As new observations accumulate, $d_{\mathcal{X}}(x, x')$ becomes smaller and the upper bound is improved. Since exploration is based on the value upper bound, NNAC encourages exploring new actions. In training, $d_{\mathcal{X}}$ is problem-specific and L can be tuned as a hyperparameter.

The distance-based bonus is less sensitive to the dimension of $\mathcal{X}$. Deep RL has poor sample efficiency for pixel data as it makes implicit use of feature encoding to find a low-dimensional embedding. In contrast, the NN approximator does not need any parametric model to capture the intrinsic states. With a proper metric defined in the high-dimensional space, the neighbors preserve their similarities and the distances can still be good indicators of their relative values.

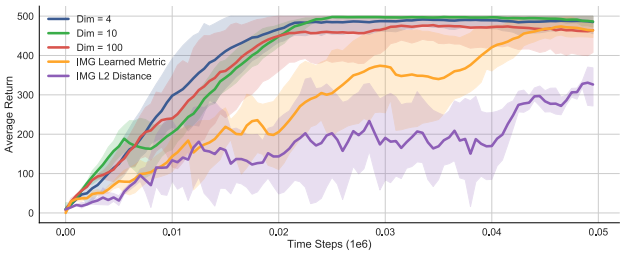
Evaluation of NNAC

We test NNAC on the cart-pole balancing problem. Due to the discrete nature of the action space, we compare with dueling double deep Q-networks (DDDQN) (Wang et al. 2016), trust region policy optimization (TRPO) (Schulman et al. 2015), proximal policy optimization (PPO) (Schulman et al. 2017), soft actor-critic (SAC) (Haarnoja et al. 2018), and neural episodic control (NEC) (Pritzel et al. 2017). Our goal is to show that the NN critic enables efficient learning with high-dimensional data, which is generally not easy to achieve. Therefore, we select a task with relatively simple dynamics and do not compare with all state-of-the-art deep RL methods that typically use more samples.

Cart-pole environment: We use the OpenAI Gym implementation (Brockman et al. 2016). The state of the cart is described by a 4-tuple $(\theta, \dot{\theta}, x, v)$, where θ and $\dot{\theta}$ are the angle and angular velocity of the pole, x is the horizontal position of the cart, and v is the velocity. The horizon and the maximum achievable reward in one episode are both 500.



(a) NNAC vs baselines with internal state descriptors



(b) NNAC with different input dimensions

Figure 1: Learning curves for CartPole-v1. The shaded areas denote one standard deviation of evaluations over 5 trails. Curves are smoothed by taking a moving average of 500 steps.

State space dimension: We prepare three types of inputs.

- $\dim(\mathcal{S}) = 4$. The 4-tuple descriptors are used directly.
- $\dim(\hat{\mathcal{S}}) = 10$ and 100. We use random projection matrices to map the 4-tuples into high-dimensional spaces. The matrix columns are orthonormal to preserve the distances and the neighbor relations in the new metric spaces.
- $\dim(\hat{\mathcal{S}}) = 4 \times 20 \times 20$. We crop the cart-pole from the 160×240 gray scale images and down-sample it to 20×20 pixels. Four consecutive frames are stacked together to derive the velocity and acceleration of the moving object.

In all cases, the L_2 distance is used for the nearest neighbor search. Considering that the L_2 distance may not be a good measurement for image similarity, we also learn a distance oracle with the Siamese network for comparison. Details of the metric learning process are provided in Appendix C.2.

Network structure and hyperparameters: We use the Stable Baselines implementation of the deep RL agents (Hill et al. 2018). For low-dimensional NNAC, we use a one-layer policy network with 32 hidden units and a ReLU activation. When learning from pixels, we add a convolutional layer with 16 units before the policy network. The discount factor γ is 0.99. The Lipschitz L is determined by a grid search and set to 7. All agents are trained with 5 random seeds. Evaluation is done every 1000 steps without exploration.

Results and discussion: Figure 1a shows the learning curves for all agents when the internal states are given. NNAC learns better policies with fewer samples than the other baselines. Also, compared with network critics that need extensive hyperparameter tuning, the NN approximator only requires experimenting with L.

Figure 1b illustrates the performance of NNAC with different input dimensions. It converges in similar steps for the intrinsic and projected states. This agrees with our proposition since the distances are preserved by the matrix transformation. For the pixel data coupled with a learned metric, NNAC achieves the same performance with slightly more samples, which might be related to learning the convolutional filters. When L_2 distance is used to measure image similarities, the algorithm is less stable and does not solve the environment within limited time steps. However, the final average return is already comparable to the deep RL agents with internal state input. Indeed, deep RL typically uses a linear output layer after several nonlinear layers. It can be interpreted as a feature encoding process followed by linear value approximation. Rather than learning the encoding, NNAC treats the distance metric as known information, thereby reducing the amount of unnecessary work.

Empirically, we find the L_2 distance to be a good choice for low-dimensional, physical state spaces, but other sophisticated metrics might be needed to capture the differences between images. For generic high-dimensional tasks, our algorithm is efficient as long as a distance oracle is provided.

The major concern of NNAC is the computational cost of finding the neighbors. We use Kd-tree (Friedman et al. 1977) in our implementation. For more complicated MDPs, the training data size can be too large to build a Kd-tree efficiently. Thus, in the next section, we introduce a new method which preserves the original neural networks of deep RL agents to reduce the computational burden.

Algorithm 2 Soft Nearest Neighbor Update

// Assume a small constant $\epsilon > 0$
Initialize policy network θ_π, value network θ_Q, $B = \emptyset$, $\alpha \leftarrow \alpha_0$
for each episode **do**
  for each environment step **do**
    Take action a according to $\pi(s)$ and exploration strategy
    Receive reward r and next state s'
    if $\alpha > \epsilon$ **then**
      $\hat{V}(s) \leftarrow \text{NNFuncApprox}(s, h, \pi, B, H)$
      $\hat{V}(s') \leftarrow \text{NNFuncApprox}(s', h+1, \pi, B, H)$
      $\delta = r + \gamma \cdot \hat{V}(s') - \hat{V}(s)$
    end if
    $B \leftarrow B \cup \{(s, a, s', r, \delta)\}$
  end for
  for each gradient step **do**
    Update the actor by Equation (12)
    Update the critic by Equation (11)
  end for
  $\alpha \leftarrow (1 - \beta) \cdot \alpha$
end for

Nearest Neighbor Update Module

In this section, we show that the NN approximator can boost the efficiency of existing deep RL algorithms. As illustrated in previous sections, our method explores the action space efficiently both in theory and in practice. However, as samples accumulate, a neural network with sufficient training data can outperform other function approximators due to its generalization ability. Hence, we propose a nearest neighbor plug-and-play module (Algorithm 2) that acts as a "starter"

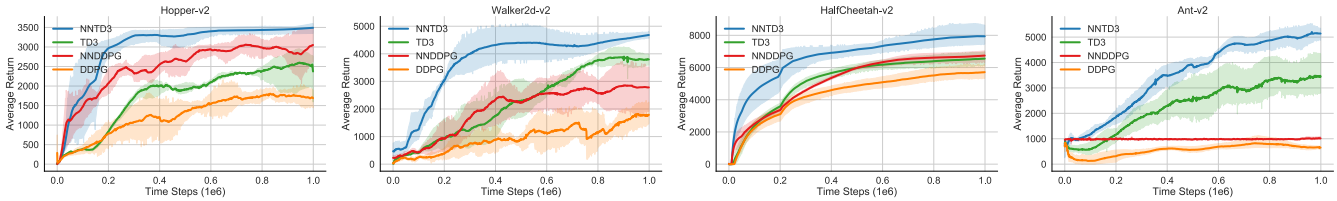


Figure 2: Learning curves for OpenAI Gym MuJoCo continuous control tasks. The shaded areas denote one standard deviation of evaluations over 5 trails. For visual clarity, the curves are smoothed by taking a moving average of 10^4 environment steps.

to accelerate deep RL agents and can be removed later.

We limit our target algorithms to actor-critic methods for simplicity of presentation. Without changing the algorithm's structure, a plug-in NN critic supplies value estimates to the rest of the framework. While the actor benefits from TD error policy gradient, the value network in the original algorithm is penalized for large deviations from the NN estimates. To better explain our idea, we include the pseudocode for two NN-augmented deep RL agents in Appendix D. We adopt an adaptive weighting scheme and gradually decrease the weight for the NN module in case it becomes a computational bottleneck. Let α be the weight of the NN approximator. There are three major components in the update module.

Modification 1: NN Value Estimation

An NN critic is incorporated to guide the training of both the actor and critic networks. In addition to the value network, NNFUNCAPPROX in Algorithm 1 is used to estimate the current state values. Assume that the original critic loss is $L(\theta_Q)$. We penalize large differences between the TD errors estimated by the value network and the NN critic. Using the mean squared error, the critic objective is reformulated as:

$$L'(\theta_Q) = (1-\alpha)\cdot L(\theta_Q) + \alpha \cdot \left\|\delta_{\theta_Q} - \delta_{NN}\right\|^2, \quad (11)$$

where $\delta_{\theta_Q} = r + \gamma V'_{\theta_Q} - V_{\theta_Q}$ is obtained from the value net, and δ_{NN} is the TD error supplied by the NN approximator.

Modification 2: TD-Regularized Policy Learning

Similar to NNAC, we also include a TD error policy gradient term in the actor loss. Let the original actor loss be $J(\theta_\pi)$. The modified gradient $\nabla_{\theta_\pi} J'(\theta_\pi)$ is:

$$(1-\alpha)\cdot \nabla_{\theta_\pi} J(\theta_\pi) + \alpha \cdot \delta_{NN} \nabla_{\theta_\pi} \log \pi(a|s;\theta_\pi). \quad (12)$$

The auxiliary TD term increases the weights for rewarding actions and decreases the weights for less preferable actions.

Modification 3: Continual TD Supervision

When the MDP has a large intrinsic dimension, it is impractical to compute the nearest neighbors at each recursion step for every mini-batch samples. Therefore, we use an exponentially decaying weight parameter for the NN critic: $\alpha = \alpha_0 \cdot (1-\beta)^k$, where $\alpha_0 \in (0, 1]$ is the initial weight, $\beta \in (0, 1)$ is the decrease rate, k is the episode number. MC simulation terminates when α is close to 0. However, past TD estimates can still supervise the learning and reduce the chance of network forgetting. This is achieved by storing δ_{NN} along with the stepwise observation in the experience buffer. For mini-batches sampled at each gradient step, (11) and (12) are used to update the networks if δ_{NN} is available. Otherwise, the original gradients are used.

Table 1: Average return over 5 trials. The top values are bolded.

Environment	NNTD3	TD3	NNDDPG	DDPG
Hopper	**3704.40**	2708.25	3202.64	2029.65
Walker2d	**5170.20**	4260.23	2917.25	2633.86
HalfCheetah	**8341.74**	7045.53	7001.52	6385.43
Ant	**4727.44**	3425.07	1115.00	1025.82

Evaluation of Soft Nearest Neighbor Update

Experiment setup: We implement the NN critic on deep deterministic policy gradient (DDPG) (Lillicrap et al. 2016) and twin delayed DDPG (TD3) (Fujimoto et al. 2018) and test with four MuJoCo locomotion tasks (Todorov et al. 2012). For fairness, we use the same hyperparameters for each method before and after adding the NN module. The network structure is selected from the benchmark work (Duan et al. 2016) and identical for all agents: a feed-forward network with two hidden layers, each with 300 and 400 units followed by a ReLU activation. L_2 distance is used to find the neighbors. MC rollout stops before 10^5 steps. Other details are given in Appendix E.

Results and discussion: Figure 2 shows the experiment results. The auxiliary NN-critic improves the sample efficiency of DDPG and TD3 in most settings, though we do not see a major performance gain for DDPG in the Ant-v2 experiment. If DDPG cannot solve an environment, the soft update module is of less help given that the original value networks still play a crucial role in learning. We summarize two principal benefits of the NN update framework.

1. NN-guided training encourages exploration and helps the agents to overcome local optima. Table 1 shows that NN algorithms obtain larger maximum returns. The Lipschitz bonus highlights exploring unvisited states. This directional exploration is more efficient than random noise.
2. The upper-bounded value estimation stabilizes training by preventing overestimation of $Q(s, a)$. This technique has a similar effect to the twin Q-networks in TD3.

Conclusion

In this paper, we provide a nearest neighbor function approximator for efficient value learning and justify its sample complexity for high-dimensional input in deterministic systems. The NN value estimator can be incorporated in model-free deep RL to encourage exploration and stabilize training. Our work suggests that there is great potential to improve deep RL with non-parametric methods. Future works can explore the benefits of nearest neighbor search in active learning or extend the theories to stochastic environments.

References

Arulkumaran, K.; Deisenroth, M.; Brundage, M.; and Bharath, A. 2017. A Brief Survey of Deep Reinforcement Learning. *ArXiv* abs/1708.05866.

Azar, M. G.; Osband, I.; and Munos, R. 2017. Minimax Regret Bounds for Reinforcement Learning. In *ICML*.

Blundell, C.; Uria, B.; Pritzel, A.; Li, Y.; Ruderman, A.; Leibo, J. Z.; Rae, J. W.; Wierstra, D.; and Hassabis, D. 2016. Model-Free Episodic Control. *ArXiv* abs/1606.04460.

Brockman, G.; Cheung, V.; Pettersson, L.; Schneider, J.; Schulman, J.; Tang, J.; and Zaremba, W. 2016. OpenAI Gym. *ArXiv* abs/1606.01540.

Buckman, J.; Hafner, D.; Tucker, G.; Brevdo, E.; and Lee, H. 2018. Sample-Efficient Reinforcement Learning with Stochastic Ensemble Value Expansion. *ArXiv* abs/1807.01675.

Busemeyer, J. R.; Byun, E.; DeLosh, E. L.; and McDaniel, M. A. 1997. Learning functional relations based on experience with input-output pairs by humans and artificial neural networks. In Lamberts, K.; and Shanks, D., eds., *Concepts and Categories*, 405–437. Cambridge: MIT Press.

Carroll, J. D. 1963. Functional Learning: The Learning of Continuous Functional Mappings Relating Stimulus and Response Continua. *ETS Research Bulletin Series* 1963(2): i–144. doi:10.1002/j.2333-8504.1963.tb00958.x.

Deisenroth, M. P.; and Rasmussen, C. E. 2011. PILCO: A Model-Based and Data-Efficient Approach to Policy Search. In *ICML*.

Duan, Y.; Chen, X.; Houthooft, R.; Schulman, J.; and Abbeel, P. 2016. Benchmarking Deep Reinforcement Learning for Continuous Control. In *ICML*.

Emigh, M.; Kriminger, E.; Brockmeier, A.; Príncipe, J.; and Pardalos, P. 2016. Reinforcement Learning in Video Games Using Nearest Neighbor Interpolation and Metric Learning. *IEEE Transactions on Computational Intelligence and AI in Games* 8: 56–66.

Friedman et al., J. 1977. An Algorithm for Finding Best Matches in Logarithmic Expected Time. *ACM Trans. Math. Softw.* 3: 209–226.

Fujimoto et al., S. 2018. Addressing Function Approximation Error in Actor-Critic Methods. *ArXiv* abs/1802.09477.

Gu, S.; Holly, E.; Lillicrap, T. P.; and Levine, S. 2017. Deep reinforcement learning for robotic manipulation with asynchronous off-policy updates. *2017 IEEE International Conference on Robotics and Automation (ICRA)* 3389–3396.

Haarnoja, T.; Zhou, A.; Abbeel, P.; and Levine, S. 2018. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. In *ICML*.

Hill, A.; Raffin, A.; Ernestus, M.; Gleave, A.; Kanervisto, A.; Traore, R.; Dhariwal, P.; Hesse, C.; Klimov, O.; Nichol, A.; Plappert, M.; Radford, A.; Schulman, J.; Sidor, S.; and Wu, Y. 2018. Stable Baselines. https://github.com/hill-a/stable-baselines.

Jaksch, T.; Ortner, R.; and Auer, P. 2008. Near-optimal Regret Bounds for Reinforcement Learning. In *J. Mach. Learn. Res.*

Jiang, N.; Krishnamurthy, A.; Agarwal, A.; Langford, J.; and Schapire, R. 2017. Contextual Decision Processes with low Bellman rank are PAC-Learnable. *ArXiv* abs/1610.09512.

Jin, C.; Allen-Zhu, Z.; Bubeck, S.; and Jordan, M. I. 2018. Is Q-learning Provably Efficient? In *NeurIPS*.

Jin, C.; Yang, Z.; Wang, Z.; and Jordan, M. I. 2020. Provably Efficient Reinforcement Learning with Linear Function Approximation. *ArXiv* abs/1907.05388.

Kansky, K.; Silver, T.; Mély, D. A.; Eldawy, M.; Lázaro-Gredilla, M.; Lou, X.; Dorfman, N.; Sidor, S.; Phoenix, D.; and George, D. 2017. Schema Networks: Zero-shot Transfer with a Generative Causal Model of Intuitive Physics. *ArXiv* abs/1706.04317.

Kégl, B. 2002. Intrinsic Dimension Estimation Using Packing Numbers. In *NIPS*.

Kober, J.; Bagnell, J.; and Peters, J. 2013. Reinforcement learning in robotics: A survey. *The International Journal of Robotics Research* 32: 1238 – 1274.

Lake, B. M.; Ullman, T. D.; Tenenbaum, J.; and Gershman, S. 2018. Building Machines That Learn and Think Like People. *The Behavioral and brain sciences* 40: e253.

Levina, E.; and Bickel, P. 2004. Maximum Likelihood Estimation of Intrinsic Dimension. In *NIPS*.

Levine, S.; Finn, C.; Darrell, T.; and Abbeel, P. 2016. End-to-End Training of Deep Visuomotor Policies. *Journal of Machine Learning Research* 17(39): 1–40.

Lillicrap, T. P.; Hunt, J. J.; Pritzel, A.; Heess, N. M. O.; Erez, T.; Tassa, Y.; Silver, D.; and Wierstra, D. 2016. Continuous control with deep reinforcement learning. *CoRR* abs/1509.02971.

Lin, Z.; Zhao, T.; Yang, G.; and Zhang, L. 2018. Episodic Memory Deep Q-Networks. *ArXiv* abs/1805.07603.

McAllister, R.; and Rasmussen, C. 2016. Improving PILCO with Bayesian Neural Network Dynamics Models.

Mnih, V.; Kavukcuoglu, K.; Silver, D.; Graves, A.; Antonoglou, I.; Wierstra, D.; and Riedmiller, M. A. 2013. Playing Atari with Deep Reinforcement Learning. *ArXiv* abs/1312.5602.

Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A. A.; Veness, J.; Bellemare, M. G.; Graves, A.; Riedmiller, M. A.; Fidjeland, A. K.; Ostrovski, G.; Petersen, S.; Beattie, C.; Sadik, A.; Antonoglou, I.; King, H.; Kumaran, D.; Wierstra, D.; Legg, S.; and Hassabis, D. 2015. Human-level control through deep reinforcement learning. *Nature* 518: 529–533.

Osband, I.; and Roy, B. V. 2014. Model-based Reinforcement Learning and the Eluder Dimension. In *NIPS*.

Plappert, M.; Houthooft, R.; Dhariwal, P.; Sidor, S.; Chen, R. Y.; Chen, X.; Asfour, T.; Abbeel, P.; and Andrychowicz, M. 2018. Parameter Space Noise for Exploration. *ArXiv* abs/1706.01905.

Pritzel, A.; Uria, B.; Srinivasan, S.; Badia, A. P.; Vinyals, O.; Hassabis, D.; Wierstra, D.; and Blundell, C. 2017. Neural Episodic Control. In *ICML*.

Schulman, J.; Levine, S.; Moritz, P.; Jordan, M.; and Abbeel, P. 2015. Trust Region Policy Optimization. In *Proceedings of the 32nd International Conference on International Conference on Machine Learning (ICML)*, 1889–1897.

Schulman, J.; Moritz, P.; Levine, S.; Jordan, M. I.; and Abbeel, P. 2016. High-Dimensional Continuous Control Using Generalized Advantage Estimation. *CoRR* abs/1506.02438.

Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal Policy Optimization Algorithms. *ArXiv* abs/1707.06347.

Shah, D.; and Xie, Q. 2018. Q-learning with Nearest Neighbors. *ArXiv* abs/1802.03900.

Silver, D.; Huang, A.; Maddison, C. J.; Guez, A.; Sifre, L.; van den Driessche, G.; Schrittwieser, J.; Antonoglou, I.; Panneershelvam, V.; Lanctot, M.; Dieleman, S.; Grewe, D.; Nham, J.; Kalchbrenner, N.; Sutskever, I.; Lillicrap, T. P.; Leach, M.; Kavukcuoglu, K.; Graepel, T.; and Hassabis, D. 2016. Mastering the game of Go with deep neural networks and tree search. *Nature* 529: 484–489.

Sun, W.; Jiang, N.; Krishnamurthy, A.; Agarwal, A.; and Langford, J. 2019. Model-based RL in Contextual Decision Processes: PAC bounds and Exponential Improvements over Model-free Approaches. In *COLT*.

Sutton, R. S. 1988. Learning to predict by the methods of temporal differences. *Machine Learning* 3: 9–44.

Todorov et al. 2012. MuJoCo: A physics engine for model-based control. *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems* 5026–5033.

van Hasselt, H.; Guez, A.; and Silver, D. 2016. Deep Reinforcement Learning with Double Q-Learning. In *AAAI*.

Wang, R.; Du, S.; Yang, L. F.; and Salakhutdinov, R. 2020. On Reward-Free Reinforcement Learning with Linear Function Approximation. *ArXiv* abs/2006.11274.

Wang, Z.; Schaul, T.; Hessel, M.; Hasselt, H. V.; Lanctot, M.; and Freitas, N. D. 2016. Dueling Network Architectures for Deep Reinforcement Learning. In *ICML*.

Yang, L. F.; Ni, C.; and Wang, M. 2019. Learning to Control in Metric Space with Optimal Regret.

Yang, L. F.; and Wang, M. 2019. Sample-Optimal Parametric Q-Learning Using Linearly Additive Features. In *ICML*.

Zanette, A.; and Brunskill, E. 2019. Tighter Problem-Dependent Regret Bounds in Reinforcement Learning without Domain Knowledge using Value Function Bounds. In *ICML*.

Theoretically Principled Deep RL Acceleration via Nearest Neighbor Function Approximation (Appendix)

Junhong Shen, Lin Yang
University of California, Los Angeles
jhshen@ucla.edu, linyang@ee.ucla.edu

A UCRL-FA

Algorithm 3 UCRL-FA (Yang, Ni, and Wang 2019)

1: **Input:** A deterministic metric MDP
2: **Initialize:** $B^{(0)} \leftarrow \emptyset$, $Q_h^{(0)}(s,a) \leftarrow H$, $\hat{r}^{(0)}(s,a) \leftarrow 1$ for all
3: $(s,a) \in \mathcal{S} \times \mathcal{A}$, $h \in [H]$
4: **for** episode $k = 1, 2, ..., K$ **do**
5: **for** step $h = 1, 2, ..., H$ **do**
6: Current state $s_h^{(k)}$
7: Play action $a_h^{(k)} = \arg\max_{a \in \mathcal{A}} Q_h^{(k)}(s_h^{(k)}, a)$
8: Record $s_{h+1}^{(k)} \leftarrow f(s_h^{(k)}, a_h^{(k)})$, $r_h^{(k)} \leftarrow r(s_h^{(k)}, a_h^{(k)})$
9: **end for**
10: $B^{(k+1)} \leftarrow B^{(k)} \cup$
11: $\big\{\big(s_i^{(k)}, a_i^{(k)}, f(s_i^{(k)}, a_i^{(k)}), r(s_i^{(k)}, a_i^{(k)})\big), i \in [H]\big\}$
12: $\hat{r}^{(k+1)} \leftarrow \textsc{FuncApprox}\big(\{(s,a), r(s,a)\}_{(s,a) \in B^{(k+1)}}\big)$
13: $Q_H^{(k+1)} \leftarrow \hat{r}^{(k+1)}$
14: Update $Q_h^{(k+1)}$ recursively:
15: $Q_h^{(k+1)} \leftarrow \textsc{FuncApprox}\big(\big\{(s,a), r(s,a) +$
16: $\sup_{a' \in \mathcal{A}} Q_{h+1}^{(k+1)}\big(f(s,a), a'\big)\big\}\big)$
17: **end for**

We present the pseudocode for upper-confidence reinforcement learning with general function approximator (UCRL-FA) proposed by Yang, Ni, and Wang (2019). In their problem setting, the MDPs have discrete state and action spaces. The rewards are assumed to be either 0 or 1. When the function approximator in Algorithm 3 takes the form of the nearest neighbor construction, Line 12 to Line 16 become:

$$\hat{r}^{(k+1)}(s,a) = \min_{(s',a') \in B^{(k+1)}} \big(r(s',a') + L_1 \cdot dist[(s,a),(s',a')]\big),$$

$$Q_H^{(k+1)} \leftarrow \min\big[\hat{r}^{(k+1)}(s,a), 1\big],$$

$$Q_h^{(k+1)} \leftarrow \min_{(s',a') \in B^{(k+1)}} \big[r(s',a') + \sup_{a'' \in \mathcal{A}} Q_{h+1}^{(k+1)}\big(f(s',a'), a''\big) + L_1 \cdot dist[(s,a),(s',a')]\big].$$

B Proof of Lemma 6

Proof. Though the state representation in $\mathcal{Y}$ is different from $\mathcal{X}$, the MDPs have the same transition model, reward function, and thus the Q-function. Let $\hat{s} = g(s) \in \mathcal{Y}$. Plug the Bi-Lipschitz condition (5) in Equations (3) and (4), we have $\forall s \in \mathcal{S}, a \in \mathcal{A}$:

$$\begin{aligned}|Q_h^*(\hat{s},a) - Q_h^*(\hat{s}',a')| &= |Q_h^*(s,a) - Q_h^*(s',a')| \\ &\le L_1 d_{\mathcal{X}}[(s,a),(s',a')] \text{ by Eq. (3)} \\ &\le L_1 C \cdot d_{\mathcal{Y}}[(\hat{s},a),(\hat{s}',a')] \text{ by Eq. (5)}\end{aligned}$$

and

$$\begin{aligned}&\max d_{\mathcal{Y}}[(f(\hat{s},a),a''),(f(\hat{s}',a'),a'')] \\ &\le C \cdot \max d_{\mathcal{X}}[(f(s,a),a''),(f(s',a'),a'')] \text{ by Eq. (5)} \\ &\le L_2 C \cdot d_{\mathcal{X}}((s,a),(s',a')) \text{ by Eq. (4)} \\ &\le L_2 C^2 \cdot d_{\mathcal{Y}}[(\hat{s},a),(\hat{s}',a')] \text{ by Eq. (5)}\end{aligned}$$

Lemma 6 follows. □

Copyright © 2021, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

C Details of the Cart-Pole Experiment

C.1 Environment Specification

An example image of the OpenAI Gym CartPole-v1 environment (Brockman et al. 2016) is presented in Figure 3. The state of the cart-pole is a 4-tuple $(\theta, \dot{\theta}, x, v)$, where θ is the vertical angle of the pole, $\dot{\theta}$ is the angular velocity, x is the horizontal position of the cart, and v is its velocity. The transition model can be described by the system of equations (Florian 2005):

$$\begin{cases} \ddot{\theta} = \dfrac{g \sin\theta - \cos\theta \frac{F + m_p l \dot{\theta}^2 \sin\theta}{m_p + m_l}}{l(\frac{4}{3} - \frac{m_p \cos\theta^2}{m_p + m_l})} \\ \ddot{x} = \dfrac{F + m_p l(\dot{\theta}^2 \sin\theta - \ddot{\theta}\cos\theta)}{m_p + m_l} \\ x = x + t \cdot \dot{x} \\ \dot{x} = \dot{x} + t \cdot \ddot{x} \\ \theta = \theta + t \cdot \dot{\theta} \\ \dot{\theta} = \dot{\theta} + t \cdot \ddot{\theta} \end{cases}$$

An episode ends when either the cart hits the track boundaries ($x \notin [-4.8, 4.8]$) or the pole has fallen over ($\theta \notin [-24°, 24°]$). The agent receives a reward 0 upon termination and $+1$ otherwise.

C.2 Distance Metric Learning

We use the Siamese network to learn the internal distance between two input image stacks. Let the images be x and x'

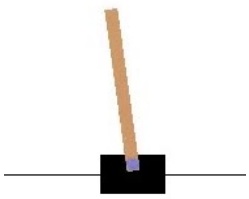


Figure 3: Image of CartPole-v1.

with internal states s and s', respectively. To obtain accurate predictions of the distances, we use larger pixel-size images with dimension $4 \times 160 \times 240$. Therefore in the experiments, we down-sample the rendered images twice with different scales: 160×240 for distance calculation and 20×20 for policy learning.

Both x and x' are first passed to a convolutional neural network individually for feature encoding. The network outputs are concatenated together and fed into a fully connected network for distance prediction. Denote the metric network as θ_d. The objective is $L(\theta_d) = \|\theta_d(x, x') - \|s - s'\|^2\|^2$. We randomly sample 100000 (x, s) pairs from the environment and train the network for 5 epochs with learning rate 1×10^{-3} and batch size 16. The network structure is:

Conv 1	128 [3×3×1] filters, leaky ReLU
Pool	2×2 Max with stride 2
Conv 2	64 [3×3×1] filters, leaky ReLU
Pool	2×2 Max with stride 2
Conv 3	16 [3×3×4] filters, leaky ReLU
Flatten	
FC	64, leaky ReLU
Concatenate	
FC	8, leaky ReLU

C.3 Hyperparameters for NNAC

NN-related parameters

- Number of nearest neighbor M: 1
- Planning horizon H': 12
- Lipschitz parameter L: 7, determined by a grid search over $\{0.1, 0.5, 1, 2, ..., 10\}$
- Action space dimension: 1
- Weight w for (s, a) pairs when calculating distance, i.e., $d(x, x') = \sqrt{\sum w_i (x_i - x'_i)^2}$:
  - Dimension 4: $(0.25, 0.25, 0.25, 0.25, 1)$
  - Dimension 10: $(0.1, ..., 0.1, 1)$
  - Dimension 100: $(0.01, ..., 0.01, 1)$
  - Image: $(1, ..., 1, 1)$

  The last dimension is the action weight.

Environment and network-related parameters

- γ: 0.99
- Policy network structure:
  - Dimension 4, 10, 100: (32, ReLU, tanh, softmax)
  - Image: (conv 16, 32, ReLU, tanh, softmax)
- Network weight initialization: $[-3 \times 10^{-3}, 3 \times 10^{-3}]$
- Optimizer: Adam
- Mini-batch size: 32
- Learning rate: 5×10^{-4}

C.4 Evaluation with Pixel Data

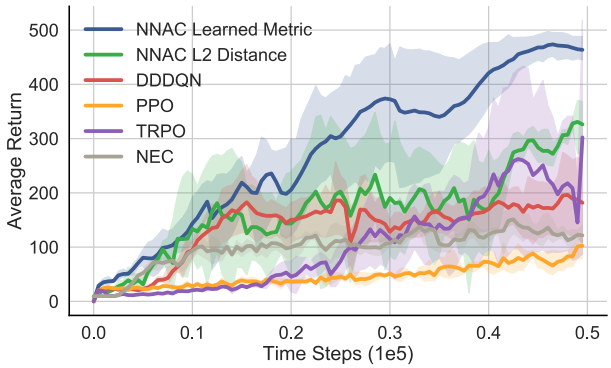


Figure 4: Learning curves for image CartPole-v1. The shaded areas denote one standard deviation of evaluations over 5 trails. For visual clarity, the curves are smoothed by taking a moving average of 500 time steps.

We also evaluate the baseline algorithms (DDDQN, PPO, TRPO, NEC) with pixel data of the CartPole-v1 environment. The image inputs are obtained in the same way as the NNAC experiment. In particular, $dim(\hat{\mathcal{S}}) = 4 \times 20 \times 20$. For the deep RL agents, we use the Stable Baselines (Hill et al. 2018) implementation with the CNN policy. For NEC, we use the author-provided implementation. The policy networks have identical structures to the NNAC actor. Figure 4 shows that compared with learning from internal state descriptors, learning from pixels requires more samples in general. NNAC with a learned metric solves the environment with the highest sample efficiency. Regardless of the distance metric used, NNAC obtains better average rewards at the end of training compared with the other algorithms.

D Example Usage of Soft NN Update Module

In deterministic control systems, unlike neural network value function approximators which may change drastically over iterations and do not have a monotonic improvement guarantee, the nearest neighbor function approximator ensures that the value upper bound becomes smaller as the data pool size becomes larger. Approximation accuracy is improved in the sense that once a regret is paid, the algorithm can gain some new information such that the same regret will not be paid again.

To better illustrate how the NN approximator can be combined with existing algorithms, we provide the pseudocode for two widely-used deep RL agents, DDPG (Lillicrap et al. 2016) and TD3 (Fujimoto et al. 2018), when equipped with the NN critic (Algorithm 4 and 5, respectively).

E Hyperparameters of MuJoCo Experiments

E.1 Soft Nearest Neighbor DDPG

NN-related parameters

Environment	Hopper	Walker2d	HalfCheetah	Ant
α_0	0.9	0.5	0.9	0.9
β	0 if $k < 20$ 1 otherwise	0 if $k < 20$ 1 otherwise	0 if $k < 20$ 1 otherwise	0.995
ϵ	10^{-3}	10^{-3}	10^{-3}	10^{-3}
M	1	1	1	1
L	7	7	5	7
H'	12	12	12	12
τ_{NN}	0.2	0.2	0.2	0.2
Neg δ scale	0.3	0.3	0.3	0.3
Grad clip	10	10	10	10

- α_0: initial NN weight.
- β: NN weight decreasing rate.
- ϵ: NN termination threshold, i.e., when NN weight is smaller than ϵ, we no longer use MC rollouts to estimate TD errors.
- M: number of nearest neighbor.
- L: Lipschitz parameter.
- H': planning horizon.
- τ_{NN} : target network update rate when using the NN critic.
- Negative TD error scale: scale the negative TD error to improve network convergence.
- Gradient clip threshold: clip the TD error policy gradient to improve network stability.

Environment and network-related parameters

Environment	Hopper	Walker2d	HalfCheetah	Ant
γ	0.99	0.99	0.99	0.99
Net. struct.	(400, ReLU, 300, ReLU)			
Weight init.	$[-3 \times 10^{-3}, 3 \times 10^{-3}]$			
Optimizer	Adam			
θ_π lr	10^{-3}	10^{-3}	10^{-3}	10^{-3}
θ_Q lr	10^{-3}	10^{-3}	10^{-3}	10^{-3}
σ_N	0.3	0.2	0.2	0.1
τ	0.005	0.005	0.005	0.005
Batch size	256	256	256	256
r scale	0.1	0.1	1	1

- γ: variance reduction parameter.
- Network structure: used in policy and value networks.
- Weight initialization: range of the uniform distribution for network weight initialization.
- Optimizer: used for both policy and value learning.
- θ_π lr: initial policy network learning rate.
- θ_Q lr: initial value network learning rate.
- σ_N: standard deviation of the normal action noise.
- τ: target networks update parameter.
- Batch size: size of mini-batches at each gradient update step.
- r scale: scale the reward to improve network stability.

E.2 Soft Nearest Neighbor TD3

NN-related parameters

Environment	Hopper	Walker2d	HalfCheetah	Ant
α_0	0.9	0.9	0.9	0.9
β	0 if $k < 20$ 1 otherwise	0 if $k < 20$ 1 otherwise	0 if $k < 20$ 1 otherwise	0 if $k < 20$ 1 otherwise
ϵ	10^{-3}	10^{-3}	10^{-3}	10^{-3}
M	1	1	1	1
L	4	4	5	4
H'	12	12	12	12
τ_{NN}	0.2	0.2	0.2	0.2
Neg δ scale	0.3	0.3	0.3	0.3
Grad clip	10	10	10	10

Environment and network-related parameters

Environment	Hopper	Walker2d	HalfCheetah	Ant
γ	0.99	0.99	0.99	0.99
Net. struct.	(400, ReLU, 300, ReLU)			
Weight init.	$[-3 \times 10^{-3}, 3 \times 10^{-3}]$			
Optimizer	Adam			
θ_π lr	10^{-3}	10^{-3}	10^{-3}	10^{-3}
θ_Q lr	10^{-3}	10^{-3}	10^{-3}	10^{-3}
σ_N	0.3	0.2	0.2	0.2
Noise clip	0.5	0.5	0.5	0.5
τ	0.005	0.005	0.005	0.005
Batch size	256	256	256	256
r scale	0.1	0.1	1	0.1
Policy freq.	2	2	2	2

- Noise clip: action noise clip threshold.
- Policy frequency: policy update frequency w.r.t. gradient update steps.

Algorithm 4 Soft Nearest Neighbor DDPG

Randomly initialize critic network θ_π and value network θ_Q
Initialize target networks $\theta'_\pi \leftarrow \theta_\pi$, $\theta'_Q \leftarrow \theta_Q$, replay buffer $B = \emptyset$, NN critic weight $\alpha \leftarrow \alpha_0$
for episode = $1, ..., K$ **do**
  Receive initial random observation s_1
  for $h = 1, ..., H$ **do**
    Take action $a_h = \pi(s_h|\theta_\pi) + N(0, \sigma)$, receive reward r_h and next state s_{h+1}
    $B \leftarrow B \cup \{(s_h, a_h, s_{h+1}, r_h, h, \mathrm{NA})\}$
    Sample a mini-batch of N transitions $(s_i, a_i, s_{i+1}, r_i, h_i, \delta_i)$
    $y_i = r_i + \gamma \cdot Q'(s_{i+1}, \pi'(s_{i+1}|\theta_{\pi'})|\theta_{Q'})$
    $\delta_i^Q = y_i - Q(s_i, a_i|\theta_Q)$
    if $\alpha > \epsilon$ **then**
      $\hat{V}(s_i) \leftarrow \mathrm{NNFuncApprox}(s_i, h_i, \pi', B, H)$
      $\hat{V}(s_{i+1}) \leftarrow \mathrm{NNFuncApprox}(s_{i+1}, h_i + 1, \pi', B, H)$
      $\delta_i^{NN} = r_i + \gamma \cdot \hat{V}(s_{i+1}) - \hat{V}(s_i)$ ▷ NN TD error estimates
      Update the critic: $L'(\theta_Q) = N^{-1} \sum \alpha \left\| \delta_i^Q - \delta_i^{NN} \right\|^2 + (1-\alpha)(y_i - Q'(s_i, a_i|\theta_{Q'}))^2$
      Update the actor: $\nabla_{\theta_\pi} J'(\theta_\pi) = N^{-1} \cdot \sum \alpha \cdot \delta_i^{NN} \nabla_{\theta_\pi} \log \pi(a_i|s_i; \theta_\pi) + (1-\alpha) \nabla_a Q(s_i, \pi(s_i|\theta_\pi)|\theta_Q) \nabla_{\theta_\pi} \pi(s_i|\theta_\pi)$
      $\delta_i \leftarrow \delta_i^{NN}$
    else
      Update the critic: $L'(\theta_Q) = N^{-1} \sum (y_i - Q'(s_i, a_i|\theta_{Q'}))^2 + \epsilon \left\| \delta_i^Q - \delta_i \right\|^2$ ▷ Continual TD supervision
      Update the actor: $\nabla_{\theta_\pi} J'(\theta_\pi) = N^{-1} \cdot \sum \nabla_a Q(s_i, \pi(s_i|\theta_\pi)|\theta_Q) \nabla_{\theta_\pi} \pi(s_i|\theta_\pi)$
    end if
    $\theta'_\pi \leftarrow \tau\theta_\pi + (1-\tau)\theta'_\pi$, $\theta'_Q \leftarrow \tau\theta_Q + (1-\tau)\theta'_Q$
  end for
  $\alpha \leftarrow (1-\beta) \cdot \alpha$
end for

Algorithm 5 Soft Nearest Neighbor TD3

Randomly initialize critic network θ_π and value networks θ_{Q_1},θ_{Q_2}
Initialize target networks $\theta'_\pi \leftarrow \theta_\pi$, $\theta'_Q \leftarrow \theta_Q$, replay buffer $B = \emptyset$, NN critic weight $\alpha \leftarrow \alpha_0$
for episode $= 1, ..., K$ **do**
  Receive initial random observation s_1
  for $h = 1, ..., H$ **do**
    Take action $a_h = \pi(s_h|\theta_\pi) + N(0, \sigma)$, receive reward r_h and next state s_{h+1}
    $B \leftarrow B \cup \{(s_h, a_h, s_{h+1}, r_h, h, \mathrm{NA})\}$
    Sample a mini-batch of N transitions $(s_i, a_i, s_{i+1}, r_i, h_i, \delta_i)$
    $y_i = r_i + \gamma \cdot \min_{n=1,2} Q'(s_{i+1}, a'_{i+1}|\theta_{Q'_n})$, $a'_{i+1} \leftarrow, \pi'(s_{i+1}|\theta_{\pi'}) + \epsilon_n, \epsilon_n \leftarrow clip\big(N(0, \sigma), -c, c\big)$
    $\delta_i^Q = y_i - \min_{n=1,2} Q'(s_i, a_i|\theta_{Q'_n})$
    if $\alpha > \epsilon$ **then**
      $\hat{V}(s_i) \leftarrow \mathrm{NNFuncApprox}(s_i, h_i, \pi', B, H)$
      $\hat{V}(s_{i+1}) \leftarrow \mathrm{NNFuncApprox}(s_{i+1}, h_i + 1, \pi', B, H)$
      $\delta_i^{NN} = r_i + \gamma \cdot \hat{V}(s_{i+1}) - \hat{V}(s_i)$
      Update the critic: $L'(\theta_{Q_n}) = \arg\min_{\theta_{Q_n}} N^{-1} \sum \alpha \left\| \delta_i^Q - \delta_i^{NN} \right\|^2 + (1-\alpha)(y_i - Q_n(s_i, a_i|\theta_{Q_n}))^2$
      if $h \mod policyfreq$ **then**
        Update the actor: $\nabla_{\theta_\pi} J'(\theta_\pi) = N^{-1} \cdot \sum \alpha \cdot \delta_i^{NN} \nabla_{\theta_\pi} \log \pi(a_i|s_i; \theta_\pi) + (1-\alpha) \nabla_a Q_1(s_i, \pi(s_i|\theta_\pi)|\theta_{Q_1}) \nabla_{\theta_\pi} \pi(s_i|\theta_\pi)$
      end if
      $\delta_i \leftarrow \delta_i^{NN}$
    else
      Update the critic: $L'(\theta_Q) = \arg\min_{\theta_{Q_i}} N^{-1} \sum (y_i - Q_i(s_i, a_i|\theta_{Q_i}))^2 + \epsilon \left\| \delta_i^Q - \delta_i \right\|^2$
      if $h \mod policyfreq$ **then**
        Update the actor: $\nabla_{\theta_\pi} J'(\theta_\pi) = N^{-1} \cdot \sum \nabla_a Q_1(s_i, a_i|\theta_{Q_1}) \nabla_{\theta_\pi} \pi(s_i|\theta_\pi)$
      end if
    end if
    $\theta'_\pi \leftarrow \tau\theta_\pi + (1-\tau)\theta'_\pi$, $\theta'_{Q_n} \leftarrow \tau\theta_{Q_n} + (1-\tau)\theta'_{Q_n}, n = 1, 2$
  end for
  $\alpha \leftarrow (1-\beta) \cdot \alpha$
end for

References

Brockman, G.; Cheung, V.; Pettersson, L.; Schneider, J.; Schulman, J.; Tang, J.; and Zaremba, W. 2016. OpenAI Gym. *ArXiv* abs/1606.01540.

Florian, R. 2005. Correct equations for the dynamics of the cart-pole system.

Fujimoto et al., S. 2018. Addressing Function Approximation Error in Actor-Critic Methods. *ArXiv* abs/1802.09477.

Hill, A.; Raffin, A.; Ernestus, M.; Gleave, A.; Kanervisto, A.; Traore, R.; Dhariwal, P.; Hesse, C.; Klimov, O.; Nichol, A.; Plappert, M.; Radford, A.; Schulman, J.; Sidor, S.; and Wu, Y. 2018. Stable Baselines. https://github.com/hill-a/stable-baselines.

Lillicrap, T. P.; Hunt, J. J.; Pritzel, A.; Heess, N. M. O.; Erez, T.; Tassa, Y.; Silver, D.; and Wierstra, D. 2016. Continuous control with deep reinforcement learning. *CoRR* abs/1509.02971.

Yang, L. F.; Ni, C.; and Wang, M. 2019. Learning to Control in Metric Space with Optimal Regret.